

Cómputo Concurrente 2026

Práctica 5

Monitores:

Candados y Condiciones

Profa: Gilde Valeria Rodríguez

1 Contexto

En las prácticas anteriores trabajamos con candados que, al no obtener el acceso, esperan en un ciclo `while` (*busy-waiting*) y consumen CPU. En el mundo real, los sistemas operativos y los lenguajes proveen objetos más sofisticados:

- **Spinlocks:** si no obtengo el candado, *quemo CPU* dando vueltas en un ciclo.
- **Monitores:** si no obtengo el candado, *me voy a dormir* y el sistema operativo me despierta cuando sea mi turno.

1.1 Monitores y variables de condición

Un **monitor** es una clase que encapsula sus variables, asegura exclusión mutua en sus métodos y permite a los hilos suspenderse esperando una condición. En Java se construye combinando un **Lock** con una o varias **Condition**.

Motivación. En el problema de productores/consumidores, un consumidor que encuentra la cola vacía no puede retener el candado, porque el productor no podría encolar (*deadlock* lógico), y tampoco puede revisar la cola sin candado sin romper la exclusión mutua. La solución es que el hilo *suelte temporalmente* el candado y se suspenda hasta que la condición se cumpla.

Una **variable de condición** es un objeto asociado a un candado que provee una *cola de espera*. Sus métodos principales son:

- `await()`: libera el candado de forma atómica y duerme al hilo. Al despertar debe re-adquirir el candado antes de continuar.
- `signal()`: despierta a *un* hilo dormido en esa condición.
- `signalAll()`: despierta a *todos* los hilos dormidos en esa condición.

Patrón obligatorio. Siempre usar `await()` dentro de un ciclo `while`, nunca con `if`:

```
1 lock.lock();
2 try {
3     while (!property) { // WHILE, nunca if
4         cond.await();
5     }
6     // la propiedad esta garantizada aqui
7     ...
8     cond.signal(); // avisa a alguien mas
9 } finally {
10     lock.unlock();
11 }
```

1.2 ¿Cómo despierta un hilo dormido?

Un hilo que despierta continúa *exactamente en la instrucción que sigue a `await()`*. La razón es que `await()` es una llamada a método más: mientras el hilo duerme, su *pila* (variables locales y dirección de retorno) y su contador de programa quedan guardados, como en cualquier cambio de contexto, y al despertar `await()` simplemente *retorna*.

```
1 while (count == 0)
2     notEmpty.await(); // aquí duerme
3 x = items[head]; // aquí continúa (tras re-evaluar el while)
```

Los pasos son:

1. **Dormir.** `await()` guarda cuántas veces el hilo tenía el candado (`holdCount`), lo libera por completo, coloca al hilo en la cola de espera de la condición y le pide al sistema operativo que lo suspenda. No consume CPU.
2. **signal().** Quien señala (con el candado) saca al hilo de la cola de la condición y lo pasa a la cola del candado. Todavía no se ejecuta.
3. **El señalizador libera el candado** en su `unlock()`.
4. **Competencia.** El hilo despertado pasa a la cola de listos y debe competir por el candado; otro hilo puede ganarle.
5. **Re-adquisición.** Al obtener el candado se le restaura el `holdCount` que tenía antes de dormir.
6. **Retorno.** `await()` termina y la ejecución sigue en la instrucción posterior.

Importante: el hilo retoma su ejecución, pero *no sabe* que la condición es verdadera. Entre el paso 2 y el paso 5 otros hilos pudieron cambiar el estado compartido, y las variables locales leídas antes del `await()` pueden estar obsoletas. Por eso `await()` va dentro de un `while`: al retornar, el programa vuelve a evaluar la condición.

1.3 Peligros

Spurious wakeups. Un hilo regresa de `await()` pero la condición ya no es válida. Ocurre, por ejemplo, cuando A está dormido, B hace `signal()` y A pasa a la cola de listos, pero antes de que A re-adquiera el candado, un hilo C entra, toma el candado y arruina la condición. El `while` permite a A volver a dormir.

Lost wakeups. Las condiciones *no tienen memoria*: si un hilo hace `signal()` y nadie está en la cola de espera, la señal desaparece para siempre. Una forma de evitarlo es usar `signalAll()`, a costa de la *thundering herd*: muchos hilos despiertan, compiten por el candado y casi todos vuelven a dormir. Otra es usar un *timeout* con `awaitNanos()`.

1.4 Candados de Lectores/Escritores

Los lectores no modifican el objeto, así que pueden estar varios a la vez. Los escritores requieren acceso exclusivo. Las reglas de seguridad son:

1. Ningún escritor entra si hay algún otro escritor o lector dentro.
2. Ningún lector entra si hay un escritor dentro.

```
1 public interface ReadWriteLock {
2     Lock readLock();
3     Lock writeLock();
4 }
```

Inanición. En `SimpleReadWriteLock`, si llegan lectores sin parar, el contador `readers` nunca llega a 0 y un escritor puede quedar bloqueado para siempre. La solución justa, `FifoReadWriteLock`, usa dos contadores monótonos (`readAcquires` y `readReleases`) y hace que el escritor “levante la mano” (`writer = true`) *antes* de esperar a que los lectores actuales se vayan, de modo que ningún lector nuevo pueda entrar.

1.5 Candados reentrantes y semáforos

Un candado **reentrante** sabe quién es su dueño. Si el mismo dueño lo adquiere otra vez, solo incrementa un contador (**holdCount**) en lugar de bloquearse a sí mismo; el candado base se libera hasta que **holdCount** vuelve a 0. Un **semáforo** (Dijkstra) generaliza el mutex: permite hasta C hilos en la sección simultáneamente. A diferencia de **signal()**, el semáforo *sí* tiene memoria: un **release()** aumenta la capacidad disponible aunque nadie esté esperando.

2 Código base

2.1 LockedQueue (monitor con dos condiciones)

```
1 class LockedQueue<T> {
2     final Lock lock = new ReentrantLock();
3     final Condition notFull = lock.newCondition();
4     final Condition notEmpty = lock.newCondition();
5     final T[] items;
6     int tail, head, count;
7
8     public void enq(T x) throws InterruptedException {
9         lock.lock();
10        try {
11            while (count == items.length)
12                notFull.await(); // duerme productor
13            items[tail] = x;
14            if (++tail == items.length) tail = 0;
15            ++count;
16            notEmpty.signal(); // avisa consumidor
17        } finally { lock.unlock(); }
18    }
19
20    public T deq() throws InterruptedException {
21        lock.lock();
22        try {
23            while (count == 0)
24                notEmpty.await(); // duerme consumidor
25            T x = items[head];
26            if (++head == items.length) head = 0;
27            --count;
28            notFull.signal(); // avisa productor
29            return x;
30        } finally { lock.unlock(); }
31    }
32 }
```

Cómo funciona LockedQueue

- Lock `lock = new ReentrantLock()` es un candado. `lock.lock()` bloquea al hilo hasta que sea el único dentro; `lock.unlock()` lo libera para que entre otro. Solo un hilo a la vez ejecuta el código que está entre ambas llamadas.
- `try { ... } finally { lock.unlock(); }` garantiza que el candado se libere aunque ocurra una excepción. Sin el `finally`, un error dejaría el candado tomado para siempre y todos los demás hilos quedarían bloqueados.
- **Condition** es una “sala de espera” asociada al candado. `await()` suelta el candado y duerme al hilo en esa sala; `signal()` despierta a uno de los que están dormidos en ella.
- **Dos condiciones.** `notFull` es la sala de los productores (esperan a que haya espacio) y `notEmpty` es la sala de los consumidores (esperan a que haya elementos). Así cada tipo de hilo espera solo lo que le interesa.
- **Cola circular.** `items` es un arreglo de tamaño fijo. `head` es la posición del siguiente elemento a sacar, `tail` la posición donde se pondrá el siguiente elemento y `count` cuántos elementos hay. Cuando un índice llega al final del arreglo, regresa a 0: `if (++tail == items.length) tail = 0`; incrementa `tail` y, si se salió del arreglo, lo reinicia.

`enq(x)`, paso a paso (productor).

1. `lock.lock()`: entra en exclusión mutua; nadie más ejecuta `enq` o `deq` en este instante.
2. `while (count == items.length) notFull.await();`: si la cola está llena, el productor suelta el candado y se duerme en `notFull`. Cuando lo despiertan vuelve a revisar la condición (por eso es `while`).

3. `items[tail] = x;` guarda el elemento.
4. `if (++tail == items.length) tail = 0;` avanza `tail` de forma circular.
5. `++count;` ahora hay un elemento más.
6. `notEmpty.signal();` avisa a un consumidor, que pudo quedarse dormido porque la cola estaba vacía.
7. `finally:` libera el candado.

deq(), paso a paso (consumidor). Es simétrico: espera mientras `count == 0` en `notEmpty`; lee `items[head]`, avanza `head` de forma circular, hace `-count`, avisa con `notFull.signal()` (ahora hay un lugar libre para un productor) y devuelve el elemento.

Ejemplo de ejecución (capacidad 2, cola vacía: `head = 0`, `tail = 0`, `count = 0`).

1. El consumidor *C* llama a `deq()`: entra, ve `count == 0`, hace `await()` y duerme en `notEmpty`. El candado queda libre.
2. El productor *P* llama a `enq(a)`: entra, guarda `items[0] = a`, `tail = 1`, `count = 1` y hace `notEmpty.signal()`: *C* sale de la sala de espera, pero aún no se ejecuta porque *P* tiene el candado.
3. *P* sale del `finally` y libera el candado. *C* lo re-adquiere, regresa de `await()` y **vuelve a evaluar** el `while`: `count = 1`, así que sale del ciclo.
4. *C* lee `a` de `items[0]`, avanza `head = 1`, `count = 0`, ejecuta `notFull.signal()` (no hay productores dormidos, la señal no tiene efecto) y devuelve `a`.

2.2 SimpleReadWriteLock

```

1 public class SimpleReadWriteLock implements ReadWriteLock {
2     int readers; // lectores activos
3     boolean writer; // escritor activo
4     Lock lock;
5     Condition condition; // monitor global
6     Lock readLock, writeLock;
7
8     public SimpleReadWriteLock() {
9         writer = false;
10        readers = 0;
11        lock = new ReentrantLock();
12        readLock = new ReadLock();
13        writeLock = new WriteLock();
14        condition = lock.newCondition();
15    }
16    public Lock readLock() { return readLock; }
17    public Lock writeLock() { return writeLock; }
18
19    class ReadLock implements Lock {
20        public void lock() {
21            SimpleReadWriteLock.this.lock.lock();
22            try {
23                while (writer) condition.await();
24                readers++;
25            } finally { SimpleReadWriteLock.this.lock.unlock(); }
26        }
27        public void unlock() {
28            SimpleReadWriteLock.this.lock.lock();
29            try {
30                readers--;
31                if (readers == 0) condition.signalAll();
32            } finally { SimpleReadWriteLock.this.lock.unlock(); }
33        }
34        // demas metodos de Lock omitidos
35    }
36

```

```

37 class WriteLock implements Lock {
38     public void lock() {
39         SimpleReadWriteLock.this.lock.lock();
40         try {
41             while (readers > 0 || writer) condition.await();
42             writer = true;
43         } finally { SimpleReadWriteLock.this.lock.unlock(); }
44     }
45     public void unlock() {
46         SimpleReadWriteLock.this.lock.lock();
47         try {
48             writer = false;
49             condition.signalAll();
50         } finally { SimpleReadWriteLock.this.lock.unlock(); }
51     }
52     // demas metodos de Lock omitidos
53 }
54 }

```

Nota: las clases internas `ReadLock` y `WriteLock` usan el candado y la condición del monitor global para modificar `readers` y `writer`. Por simplicidad se omite el manejo de `InterruptedException`; en su implementación deben manejarlo.

Cómo funciona SimpleReadWriteLock

La idea. Muchos lectores pueden leer a la vez, pero un escritor necesita estar solo. Para saber quién está adentro el monitor lleva dos datos: `readers` (cuántos lectores hay dentro) y `writer` (si hay un escritor dentro). Todos los hilos que deben esperar se duermen en una sola sala de espera, `condition`.

Dos candados distintos. Es el punto que más confunde:

- El **candado interno** (`lock`, un `ReentrantLock`) solo protege a `readers` y `writer` durante unos instantes. Se toma y se suelta dentro de cada método.
- El **candado de lectura/escritura** (`readLock()` y `writeLock()`) es el que el usuario mantiene “tomado” mientras lee o escribe. Cuando `rw.readLock().lock()` regresa, el candado interno ya fue liberado; lo que queda es una anotación en `readers` o `writer` que indica quién está dentro.

```

1 ReadWriteLock rw = new SimpleReadWriteLock();
2 rw.readLock().lock();
3 try { /* leer el recurso compartido */ }
4 finally { rw.readLock().unlock(); }

```

Clases internas. `ReadLock` y `WriteLock` están declaradas dentro de `SimpleReadWriteLock` para poder modificar `readers` y `writer`. Como ambas tienen un método llamado `lock()`, se escribe `SimpleReadWriteLock.this.lock` para dejar claro que nos referimos al candado interno del monitor externo.

`ReadLock.lock()`. Toma el candado interno y espera con `while (writer) condition.await()`; mientras haya un escritor dentro. Cuando no lo hay, hace `readers++` (“hay un lector más”) y suelta el candado interno.

`ReadLock.unlock()`. Hace `readers--`. Si era el último lector (`readers == 0`), llama a `signalAll()`, porque los únicos que pueden estar esperando eso son los escritores.

`WriteLock.lock()`. Espera con `while (readers > 0 || writer)`: no entra si hay lectores *ni* si hay otro escritor. La segunda parte es necesaria: dos escritores podrían ver `readers == 0` al mismo tiempo y entrar juntos si no se revisara `writer`. Al entrar hace `writer = true`.

`WriteLock.unlock()`. Hace `writer = false` y `signalAll()`. Se despierta a todos porque en la misma sala duermen lectores y escritores y no se sabe cuál puede avanzar; cada uno re-evalúa su propio `while` y el que no puede vuelve a dormir.

Ejemplo de ejecución.

1. Los lectores L_1 y L_2 llaman a `readLock().lock()`: ambos ven `writer == false` y entran (`readers = 2`).
2. El escritor W llama a `writeLock().lock()`: ve `readers > 0` y se duerme en `condition`.

3. L_1 sale (`readers = 1`, no hay señal). L_2 sale (`readers = 0`) y hace `signalAll()`.
4. W despierta, re-evalúa su `while (readers == 0 y writer == false)`, entra y pone `writer = true`.
5. Un lector L_3 que llega ahora ve `writer == true` y duerme. Cuando W sale, `writer = false` y `signalAll()` lo despierta.

El problema. Si llegan lectores sin parar, `readers` nunca baja a 0 y W duerme indefinidamente (inanición). Esto lo corrige `FifoReadWriteLock`.

Cómo funciona `FifoReadWriteLock`

El código completo aparece en el Ejercicio 2; aquí se explica su lógica.

Dos contadores que solo suben. En lugar de `readers` se usan `readAcquires` (cuántos lectores han entrado en total) y `readReleases` (cuántos han salido en total). Los lectores que hay dentro son la diferencia entre ambos, así que `readAcquires == readReleases` significa “no hay lectores”.

La bandera `writer` significa “hay un escritor dentro o un escritor que ya levantó la mano y espera a que salgan los lectores actuales”.

`ReadLock.lock()`.

```
1 while (writer) condition.await(); // si un escritor levanto la mano, espero
2 readAcquires++;
```

`WriteLock.lock()` tiene tres pasos:

```
1 while (writer) condition.await(); // 1. otros escritores
2 writer = true; // 2. levanto la mano
3 while (readAcquires != readReleases) // 3. espero a los lectores actuales
4     condition.await();
```

El paso 2 es la diferencia clave: `writer = true` se pone *antes* de esperar a los lectores. Desde ese momento, cualquier lector nuevo ve `writer == true` y espera, así que solo quedan los lectores que ya estaban adentro, y al salir llevan `readReleases` a igualarse con `readAcquires`.

`ReadLock.unlock()`. Hace `readReleases++` y, si ya no quedan lectores (`readAcquires == readReleases`), hace `signalAll()` para despertar al escritor que espera.

`WriteLock.unlock()`. Hace `writer = false` y `signalAll()`, para que entren los lectores y escritores que esperaban.

Ejemplo de ejecución (compara con el de `SimpleReadWriteLock`).

1. El lector L_1 entra: `readAcquires = 1`, `readReleases = 0`.
2. El escritor W llega: `writer == false`, así que pone `writer = true`. Como `readAcquires != readReleases`, hace `await()`.
3. El lector L_2 llega después: ve `writer == true` y se duerme. (En `SimpleReadWriteLock` habría entrado y W seguiría esperando.)
4. L_1 sale: `readReleases = 1`, ahora son iguales, y hace `signalAll()`. Despiertan W y L_2 .
5. Si L_2 obtiene primero el candado interno, ve `writer == true` y vuelve a dormir. W re-evalúa su segundo `while`, los contadores son iguales, y entra a escribir.
6. Cuando W sale, `writer = false` y `signalAll()` despierta a L_2 , que ahora sí puede entrar.

Con esto ningún escritor sufre inanición: los lectores que llegan después de que él levanta la mano esperan su turno.

3 Ejercicios

Instrucciones:

- Entrega en un **PDF** las respuestas de los ejercicios teóricos. En los ejercicios que requieran implementación, añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Solo un integrante del equipo debe subir la práctica. Los demás integrantes deben marcar la tarea como entregada y escribir, en un comentario privado dentro de la práctica, el nombre completo de la persona que realizó la entrega.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Recuerda que debes utilizar **Java 21 LTS**.
- Si no compila utilizando Java 21 LTS o si no se entrega la breve descripción de los programas se penalizará.

Tiempo de elaboración: ≈ 2 hr

Total de puntos: 100

1. [15 min / 20 puntos] *Spurious wakeups*. Un compañero reescribe el método `deq()` de `LockedQueue` cambiando el `while` por un `if`:

```
1 if (count == 0) notEmpty.await();
```

- (a) (6 pts) Describe, paso a paso, una ejecución con un productor P y dos consumidores C_1 y C_2 en la que C_1 termina ejecutando el resto de `deq()` con `count == 0`. Indica en cada paso quién tiene el candado y el valor de `count`.
- (b) (4 pts) ¿Qué valor termina teniendo `count` y qué le pasa a la cola? Explica cómo el `while` evita el problema.

2. [30 min / 40 puntos] Considera el monitor *FifoReadWriteLock*, actualmente cumple con:

- (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica.
- (b) Si un escritor está en su sección crítica, **ningún escritor o lector** puede entrar a su propia sección crítica.

```
1 public class FifoReadWriteLock implements ReadWriteLock {
2     int readAcquires; // entradas
3     int readReleases; // salidas
4     boolean writer;
5     Lock lock;
6     Condition condition;
7     Lock readLock, writeLock;
8
9     public FifoReadWriteLock() {
10         readAcquires = 0; readReleases = 0;
11         writer = false;
12         lock = new ReentrantLock();
13         condition = lock.newCondition();
14         readLock = new ReadLock();
15         writeLock = new WriteLock();
16     }
17     public Lock readLock() { return readLock; }
18     public Lock writeLock() { return writeLock; }
19
20     class ReadLock implements Lock {
21         public void lock() {
22             FifoReadWriteLock.this.lock.lock();
23             try {
24                 while (writer) condition.await();
```

```

25         readAcquires++;
26     } finally { FifoReadWriteLock.this.lock.unlock(); }
27 }
28 public void unlock() {
29     FifoReadWriteLock.this.lock.lock();
30     try {
31         readReleases++;
32         if (readAcquires == readReleases) condition.signalAll();
33     } finally { FifoReadWriteLock.this.lock.unlock(); }
34 }
35 }
36
37 class WriteLock implements Lock {
38     public void lock() {
39         FifoReadWriteLock.this.lock.lock();
40         try {
41             while (writer) condition.await(); // 1. otros escritores
42             writer = true; // 2. levanto la mano
43             while (readAcquires != readReleases) // 3. lectores actuales
44                 condition.await();
45         } finally { FifoReadWriteLock.this.lock.unlock(); }
46     }
47     public void unlock() {
48         FifoReadWriteLock.this.lock.lock();
49         try {
50             writer = false;
51             condition.signalAll();
52         } finally { FifoReadWriteLock.this.lock.unlock(); }
53     }
54 }
55 }

```

Implementa el monitor modificando la propiedad (b) de forma que ahora se cumpla:

- (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica.
- (b) Si un escritor está en su sección crítica, **ningún lector** puede entrar a su propia sección crítica.

Entrega la clase y un programa de prueba, y explica brevemente por qué tu implementación cumple (a) y (b) y por qué ahora pueden coexistir varios escritores.

3. [30 min / 40 puntos] Considera el monitor *SimpleReadWriteLock* de la sección de código base, actualmente cumple con:

- (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica.
- (b) Si un escritor está en su sección crítica, ningún escritor o lector puede entrar a su propia sección crítica.
- (c) Cualquier cantidad de lectores puede estar simultáneamente en su sección crítica.

Implementa el monitor modificando la propiedad (c) de forma que ahora se cumpla:

- (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica.
- (b) Si un escritor está en su sección crítica, ningún escritor o lector puede entrar a su propia sección crítica.
- (c) **A lo más K lectores** pueden estar simultáneamente en su sección crítica, donde K es un parámetro del constructor.

No es necesario que el protocolo sea justo. Responde además: ¿es suficiente que `ReadLock.unlock()` llame a `signalAll()` solo cuando `readers == 0`? Justifica con una ejecución.

References

- [1] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann Publishers Inc., 2008. Cap. 8: Monitors and Blocking Synchronization.

- [2] Oracle. *Java SE Documentation — Interface Condition*. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/locks/Condition.html>
- [3] JSR-166: Concurrency Utilities. `ReentrantReadWriteLock`.